

DEVOPS MASTER COURSE

Volume 8E

Google Cloud Enterprise DevOps, Observability, Security & Governance

Chapters 41-55

*Production infrastructure as code, secure delivery, observability, threat detection,
governance, FinOps, labs, troubleshooting, and enterprise capstone*

2026 Edition

About This Part

Volume 8E turns Google Cloud knowledge into a governed enterprise operating model. The focus is not simply to create resources, execute builds, or enable security products. The goal is to make infrastructure, delivery, observability, access, evidence, and cost controls repeatable under normal operations and dependable during incidents.

The chapters move from Terraform and managed infrastructure deployment into platform products, continuous integration and progressive delivery, software-supply-chain controls, telemetry, audit, cryptography, security posture management, security operations, data perimeters, landing-zone governance, reliability, and FinOps. Every topic is framed as a production system with owners, failure modes, controls, evidence, and recovery procedures.

Volume 8 Progress

Part	Coverage	Status
8A	Google Cloud Foundations	Complete
8B	Core Infrastructure, Connectivity, Storage, Backup, and DR	Complete
8C	Databases, Analytics, Data Engineering, and Migration	Complete
8D	Containers, Serverless, APIs, Events, and Workflows	Complete
8E	DevOps, IaC, Observability, Security, Governance, and FinOps	This part
8F	Enterprise Projects, Certifications, Troubleshooting, and Interviews	Next

Scope, Safety, and Cost Controls

- Use isolated training projects, budgets, quotas, short retention periods, and explicit deletion steps. Build minutes, private pools, log ingestion, monitoring, vulnerability scanning, security services, KMS, NAT, and data exports can create ongoing charges.
- Use short-lived federation and attached service identities. Do not place service-account keys, OAuth tokens, private keys, secrets, or production configuration in repositories, build logs, images, Terraform state, or pipeline variables.
- Apply organization policies, deny rules, service perimeters, firewall policies, and security-response automation first in a test folder or canary project. A governance error can block deployment, monitoring, recovery, or administrative access.
- Treat Terraform state, build artifacts, provenance, audit logs, findings, and incident evidence as sensitive records. Restrict access, encrypt them, define retention, and test recovery.
- Validate current product availability, service tiers, quotas, regional support, pricing, runtime support, and release notes in official Google Cloud documentation before production adoption.

Table of Contents

Chapter 41 Terraform on Google Cloud and Infrastructure Manager

Chapter 42 Terraform Blueprints, Modules, and Platform Engineering

Chapter 43 Cloud Build, Secure Source Management, and Build Infrastructure

Chapter 44 Cloud Deploy and Progressive Delivery

Chapter 45 CI/CD Security, Provenance, and Policy Enforcement

Chapter 46 Cloud Monitoring, Managed Prometheus, SLOs, and Alerting

Chapter 47 Cloud Logging, OpenTelemetry, Trace, and Application Monitoring

Chapter 48 Cloud Audit Logs, Asset Inventory, Policy Intelligence, and Fleet Operations

Chapter 49 Secret Manager, Cloud KMS, HSM, EKM, and Certificate Lifecycle

Chapter 50 Security Command Center and Cloud Security Posture Management

Chapter 51 Google Security Operations and Detection Engineering

Chapter 52 VPC Service Controls, Cloud Armor, Firewall Policies, and Zero Trust

Chapter 53 Landing Zones, Organization Policy, and Governance at Scale

Chapter 54 Well-Architected, Reliability Engineering, Sustainability, and FinOps

Chapter 55 Enterprise Google Cloud Platform and Secure Delivery Capstone

Enterprise Practice Projects

Senior Interview Drill Bank

Google Cloud DevOps Command Reference

Production Readiness Checklist

Glossary

Official Google Cloud Documentation Index

Chapter 41 - Terraform on Google Cloud and Infrastructure Manager

Declarative provisioning, state control, plan review, managed deployments, and drift-aware operations

Learning Objectives

- Explain Terraform providers, resources, data sources, modules, state, plans, and dependency graphs.
- Design secure remote state, provider authentication, project boundaries, and promotion workflows.
- Use Infrastructure Manager to preview and deploy Terraform blueprints through a managed Google Cloud service.
- Operate imports, drift correction, replacement risk, rollback, and controlled destruction.

Why This Matters in Production

Terraform on Google Cloud and Infrastructure Manager is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

41.1 Terraform Execution Model

Terraform evaluates configuration and provider schemas, reads current state, refreshes remote objects, creates a dependency graph, and produces a plan. The plan is a proposed transition, not a guarantee: APIs can change, quotas can be exhausted, policies can deny actions, and concurrent actors can modify resources between planning and application. Production workflows therefore lock state, pin providers, review replacement actions, validate policy, and re-plan immediately before an approved apply.

41.2 State, Authentication, and Trust Boundaries

State can contain resource identifiers, topology, outputs, and sensitive values. Store it in a restricted Cloud Storage bucket with versioning, retention appropriate to recovery needs, uniform bucket-level access, and logging. Use Workload Identity Federation or attached identities for automation instead of service-account keys. Separate the identity that proposes changes from the identity that applies them, and scope permissions to the projects and services managed by each stack.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

41.3 Infrastructure Manager

Infrastructure Manager deploys Terraform configurations, called blueprints, through a managed Google Cloud toolchain. It can preview, create, update, and delete deployments while keeping deployment records inside Google Cloud. It reduces the need to maintain a dedicated Terraform execution platform, but teams still own module quality, state boundaries, IAM, policy checks, change approval, secrets handling, and recovery decisions.

41.4 Change Safety and Recovery

A safe workflow classifies plan actions into create, in-place update, replacement, and destroy. Stateful replacements require backups, migration procedures, and explicit approval. Import is used to bring existing resources under management; moved blocks and state moves preserve identity during refactoring. Failed applies are investigated from both Terraform output and service audit logs. Rollback usually means applying a known-good configuration, not reversing every API call automatically.

Production Design Checklist

- Explain Terraform providers, resources, data sources, modules, state, plans, and dependency graphs.

- Design secure remote state, provider authentication, project boundaries, and promotion workflows.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Plan wants to recreate a resource	Address, immutable property, provider change, or state mismatch	Inspect plan and provider notes; use moved/import operations or redesign before apply
State lock persists	Interrupted run or competing automation	Confirm no active writer, preserve state backup, then release lock through the supported backend process
Apply denied	IAM, organization policy, quota, service perimeter, or API disabled	Use Policy Troubleshooter, audit logs, quota data, and API status; do not broaden roles blindly
Drift returns after correction	External automation or manual owner keeps changing resource	Identify the competing controller, establish ownership, and remove duplicate writers

Hands-on Lab 41: Deploy a Reviewed Terraform Blueprint

Prerequisites

- A sandbox project, Cloud Storage state bucket, Terraform, gcloud, and an identity authenticated without a downloaded key.

Implementation

1. Create a small root module that provisions a labeled storage bucket and log sink or another low-cost resource.
2. Configure a protected remote state bucket and verify that the automation identity has only required access.
3. Run `fmt`, `validate`, `provider` initialization, and a saved plan; inspect every create, update, replace, and destroy action.
4. Create a deliberate out-of-band label change, refresh state, and document the detected drift.
5. Restore the intended value through configuration rather than through another manual edit.
6. Package the root module as an Infrastructure Manager blueprint, run a preview, and compare it with the local plan.
7. Deploy, capture deployment evidence, then destroy the lab resources through the managed configuration.

Validation Evidence

- No long-lived service-account key exists.
- State versions can be recovered and are not publicly accessible.
- The drift and replacement review is documented.
- Local and managed deployment records identify the same intended resources.

Cleanup

- Delete the Infrastructure Manager deployment, training resources, temporary identities, and state bucket after preserving only non-sensitive evidence.

Interview Questions and Model Answers

1. Why is a Terraform plan not a guarantee?

It is calculated from a point-in-time view; policies, quotas, provider behavior, concurrent changes, or API conditions can differ when apply occurs.

2. Why protect state?

State is the coordination record for managed objects and can expose topology or sensitive values; corruption or unauthorized modification can cause destructive actions.

3. When is Infrastructure Manager useful?

When an organization wants a managed Google Cloud execution and deployment record for Terraform blueprints rather than operating its own runners and state workflow.

4. How should failed applies be recovered?

Preserve state, inspect partial changes and audit logs, correct the configuration or environment, re-plan, and apply deliberately rather than editing state casually.

Chapter Summary

- Terraform production safety depends on state integrity, short-lived identity, reviewed plans, and controlled ownership.
- Infrastructure Manager provides a managed deployment toolchain but does not replace architecture and governance decisions.
- Imports, refactors, drift, and replacement actions require explicit lifecycle procedures.
- Recovery is based on evidence and a new approved plan.

Official Google Cloud References

<https://docs.cloud.google.com/docs/terraform>

<https://docs.cloud.google.com/infrastructure-manager/docs/overview>

<https://docs.cloud.google.com/infrastructure-manager/docs/preview-deployment>

Chapter 42 - Terraform Blueprints, Modules, and Platform Engineering

Reusable infrastructure products, version contracts, policy guardrails, testing, and developer experience

Learning Objectives

- Design modules around ownership, lifecycle, security, and blast-radius boundaries.
- Publish versioned blueprints with inputs, outputs, validation, examples, and migration guidance.
- Build platform products that combine infrastructure, policy, observability, documentation, and support.
- Measure platform adoption, reliability, exception volume, and developer outcomes.

Why This Matters in Production

Terraform Blueprints, Modules, and Platform Engineering is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

42.1 Modules as Product Contracts

A module should express a stable capability such as a project factory, private service, governed bucket, GKE workload landing zone, or observability baseline. Inputs are a public API. Defaults must be secure and understandable; validation should reject unsafe combinations; outputs should expose only supported integration points. Internal resource names and implementation details should remain replaceable so the platform team can evolve the module without forcing consumers to understand every service API.

42.2 Blueprint Composition and Release Management

Blueprints compose modules into deployable environments. Pin module and provider versions, generate release notes, use semantic versioning where it reflects compatibility, and publish upgrade paths. Breaking changes require deprecation windows and migration testing. A golden path should make the compliant route easier than manual provisioning, while still supporting documented exception workflows for genuinely different workloads.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

42.3 Testing and Policy Guardrails

Quality gates include formatting, static validation, unit tests for expressions, integration tests in disposable projects, policy-as-code checks, cost estimation, security scanning, and plan review. Tests must cover failure and deletion paths, not only successful creation. Organization Policy and IAM remain authoritative controls; module checks give earlier feedback and more useful error messages before platform APIs reject a deployment.

42.4 Platform Operating Model

Platform engineering is a service relationship. Product teams need versioned templates, documentation, examples, SLOs, release channels, support ownership, and a roadmap. Measure lead time to a compliant environment, failed deployment rate, time to recover, exception volume, support demand, cost per environment, and developer satisfaction. Remove unused variants rather than accumulating permanent bespoke paths.

Production Design Checklist

- Design modules around ownership, lifecycle, security, and blast-radius boundaries.
- Publish versioned blueprints with inputs, outputs, validation, examples, and migration guidance.

- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Module upgrade changes many resources	Hidden implementation coupling or unpinned dependency	Review module diff and upgrade guide; test in disposable environment; split breaking change
Consumers bypass module	Golden path missing capability or too difficult	Analyze demand, add supported extension or document exception instead of relying only on enforcement
Integration tests are flaky	Shared project, eventual consistency, quota, or non-unique names	Use disposable projects, deterministic names, bounded retries, cleanup verification, and quota isolation
Policy test passes but deployment fails	Test coverage differs from organization policy or runtime context	Export effective policies and add integration test using the target folder/project hierarchy

Hands-on Lab 42: Publish a Versioned Project-Factory Blueprint

Prerequisites

- Terraform, a sandbox organization or folder, test projects, and permissions to create low-cost project-level resources.

Implementation

1. Define a module contract for a workload project: labels, APIs, service identities, logging, budget linkage, and optional Shared VPC attachment.
2. Implement input validation and secure defaults; reject missing owners and prohibited regions.
3. Create examples for development and production variants without duplicating module internals.
4. Add formatting, validation, policy, and integration tests in a disposable test project.
5. Publish version 1.0 with release notes, support owner, SLO, and upgrade policy.
6. Introduce a backward-compatible enhancement and demonstrate a non-disruptive upgrade.
7. Create a deliberate breaking change proposal and document deprecation and migration instead of releasing it immediately.

Validation Evidence

- Consumers can deploy without editing module internals.
- Unsafe inputs fail before production creation.
- Integration tests create and clean all expected resources.
- Version and migration evidence is reproducible.

Cleanup

- Delete disposable projects and temporary build artifacts; retain only module source and sanitized test results.

Interview Questions and Model Answers

1. What makes a good Terraform module boundary?

A stable capability with one owner and lifecycle, limited blast radius, clear inputs and outputs, and replaceable internals.

2. Why is a secure default insufficient?

Consumers can override it or use the module in a different context; validation, policy enforcement, documentation, and monitoring are also required.

3. How do platform teams prevent template sprawl?

Publish a small supported catalog, measure usage, deprecate variants, and convert repeated exceptions into deliberate products only when justified.

4. What should a platform product include besides code?

Documentation, examples, ownership, support, SLOs, release process, security controls, observability, cost expectations, and migration guidance.

Chapter Summary

- Modules are product contracts, not copied snippets.
- Blueprints need versioning, tests, policy checks, and migration discipline.
- Golden paths succeed when they reduce cognitive load and operational risk.
- Platform performance is measured through user and reliability outcomes.

Official Google Cloud References

<https://docs.cloud.google.com/docs/terraform/blueprints/terraform-blueprints>

<https://docs.cloud.google.com/docs/terraform/best-practices-for-terraform>

<https://cloud.google.com/architecture/landing-zones>

Chapter 43 - Cloud Build, Secure Source Management, and Build Infrastructure

Hermetic builds, triggers, private pools, caching, provenance, and secure build operations

Learning Objectives

- Design Cloud Build configurations, triggers, substitutions, artifacts, and build identities.
- Choose hosted workers or private pools based on connectivity, isolation, performance, and compliance.
- Create reproducible builds with pinned dependencies, bounded caches, and test evidence.
- Operate failures, quotas, logs, artifact retention, and build-platform security.

Why This Matters in Production

Cloud Build, Secure Source Management, and Build Infrastructure is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

43.1 Build Pipeline Architecture

Cloud Build executes a directed sequence of containerized build steps. A production pipeline separates source checkout, dependency resolution, compilation, tests, security checks, packaging, publishing, and provenance. Each step receives only the files and credentials it requires. The build service account is not a general administrator; use separate identities or impersonation for publishing and deployment. Triggers must bind repositories, branches, tags, and pull-request events to explicit review rules.

43.2 Source, Reproducibility, and Dependency Trust

A build is reproducible when the same source, declared toolchain, dependency lock files, and parameters create equivalent outputs. Pin builder images by digest where practical, verify checksums, restrict external package access, and use approved mirrors. Secure Source Manager or an approved external source platform should enforce branch protection, review, signed commits or equivalent evidence, and least-privilege automation access.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

43.3 Hosted Workers and Private Pools

Default workers are appropriate for many internet-connected builds. Private pools provide dedicated worker pools, VPC connectivity, configurable machine types, and stronger network control for private dependencies or regulated workloads. They also require capacity, quota, egress, DNS, patch, cost, and scaling decisions. Do not create a private pool merely to hide an overprivileged build identity.

43.4 Build Performance and Operations

Caching can reduce time but can also preserve compromised or stale dependencies. Separate immutable artifact caches from mutable workspace caches, set retention, and include cache invalidation in incident response. Use build timeouts, concurrency limits, log routing, failure notifications, and artifact lifecycle rules. Track queue time, build duration, test flakiness, failure reasons, and cost per successful artifact.

Production Design Checklist

- Design Cloud Build configurations, triggers, substitutions, artifacts, and build identities.
- Choose hosted workers or private pools based on connectivity, isolation, performance, and compliance.
- Identity and permissions use least privilege and short-lived credentials.

- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Build cannot access private dependency	Private pool routing, DNS, firewall, proxy, or service perimeter	Trace DNS and routes from worker pool; validate egress and perimeter policy
Trigger uses wrong revision	Broad branch pattern, tag confusion, or mirrored repository lag	Inspect trigger event and resolved commit; narrow filters and record immutable revision
Build succeeds locally but fails remotely	Undeclared dependency, architecture difference, missing credential, or workspace assumption	Make toolchain explicit, containerize build environment, pin dependencies, and remove local state
Queue time rises	Private pool capacity, regional shortage, or concurrency quota	Measure queue versus execution time; scale pool or redistribute workloads with quotas and cost controls

Hands-on Lab 43: Build a Reproducible Artifact with a Private Dependency Check

Prerequisites

- A sandbox project, source repository, Artifact Registry repository, Cloud Build, and optional small private pool.

Implementation

1. Create a minimal application with dependency lock file, unit tests, and a multi-stage container build.
2. Write a Cloud Build configuration with separate test, scan, build, and publish steps.
3. Use a dedicated build identity with pull access to approved dependencies and write access only to the target repository.
4. Pin the base image and record source revision, builder image digest, build parameters, and output digest.
5. Configure a trigger requiring the protected main branch and a separate pull-request validation trigger.
6. Run one build with a deliberately undeclared dependency, capture failure, then make the dependency explicit.
7. Compare hosted-worker and private-pool behavior or document the decision not to use a private pool.

Validation Evidence

- The output digest is traceable to an immutable source revision.
- A build identity cannot deploy production workloads.
- The undeclared dependency failure is reproducible.
- Queue, duration, logs, tests, and scan evidence are retained.

Cleanup

- Delete test triggers, builds if required by policy, images, temporary pools, and source projects.

Interview Questions and Model Answers

1. Why separate build and deploy identities?

It limits blast radius and prevents compromise of one pipeline stage from creating and releasing unreviewed software.

2. When is a private pool justified?

When builds require private network access, dedicated capacity or isolation, specific machine characteristics, or compliance controls that hosted workers cannot meet.

3. What is a hermetic build?

A build whose inputs and toolchain are explicitly controlled so hidden host state or untrusted network dependencies do not change the output.

4. Why can caching be a security risk?

A mutable cache can preserve poisoned or outdated dependencies and bypass normal verification unless integrity and invalidation are controlled.

Chapter Summary

- Cloud Build pipelines should create traceable artifacts from explicit inputs.
- Source controls, build identity, worker network, dependencies, and caches are all trust boundaries.
- Private pools solve network and isolation requirements but introduce operating cost.
- Build-platform SLOs include queue time, reliability, reproducibility, and evidence completeness.

Official Google Cloud References

<https://docs.cloud.google.com/build/docs>

<https://docs.cloud.google.com/build/docs/private-pools/private-pools-overview>

<https://docs.cloud.google.com/secure-source-manager/docs/overview>

Chapter 44 - Cloud Deploy and Progressive Delivery

Release pipelines, targets, approvals, canary rollout, verification, rollback, and environment promotion

Learning Objectives

- Model delivery pipelines, targets, releases, rollouts, approvals, and environment ownership.
- Implement progressive delivery with canary phases, verification gates, and bounded blast radius.
- Separate artifact creation from release authorization and environment credentials.
- Design rollback, roll-forward, incident pause, and release evidence procedures.

Why This Matters in Production

Cloud Deploy and Progressive Delivery is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

44.1 Release Model and Separation of Duties

Cloud Deploy manages delivery progression after an artifact has been built. A release binds a versioned delivery configuration to immutable artifacts; rollouts move that release to targets. This separation prevents every build from becoming a production deployment. Environment owners define target access, approvals, maintenance windows, policy, and verification, while application teams provide release metadata and health expectations.

44.2 Progressive Delivery

Canary deployment exposes a new version to a limited share of traffic or capacity, observes it, and expands only when evidence is acceptable. Phase percentages, bake time, metrics, logs, error budgets, and manual approvals must reflect business risk. A 5 percent canary is not low risk when the affected users are high-value tenants or when all background jobs use the new version.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

44.3 Verification and Promotion

A release gate should combine pre-deployment validation, deployment health, functional smoke tests, security policy, and post-deployment service indicators. Verification must query the same user-visible signals used during incidents. Promotions across development, staging, and production should reuse the same artifact digest while changing only approved environment configuration.

44.4 Rollback and Incident Control

Rollback is fast only when old artifacts, configuration, schemas, and infrastructure remain compatible. Database changes need expand-contract sequencing. Operators need permission to pause, reject, retry, or roll back without obtaining broad project administration. Every production rollout should record initiator, artifact digest, configuration, approvals, target, timing, health verdict, and recovery action.

Production Design Checklist

- Model delivery pipelines, targets, releases, rollouts, approvals, and environment ownership.
- Implement progressive delivery with canary phases, verification gates, and bounded blast radius.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Canary looks healthy but users fail	Wrong metric scope, synthetic coverage, tenant skew, or delayed symptom	Add user and business signals; segment by region/tenant; extend bake time
Rollback fails	Schema/config incompatibility or old artifact removed	Use backward-compatible migrations, retain artifacts, and test rollback before production
Rollout cannot access target	Service identity, target IAM, network, or cluster authorization	Verify deployment identity chain and target-specific permissions without broadening project roles
Approval blocks emergency fix	Approval design lacks break-glass process	Use time-bound, audited emergency path with post-incident review, not permanent bypass

Hands-on Lab 44: Operate a Canary Release with Automated Verification

Prerequisites

- A Cloud Run service or small GKE application, Artifact Registry image, Cloud Deploy, and monitoring metrics.

Implementation

1. Create development, staging, and production targets with separate deployment identities.
2. Build and publish one immutable image digest; create a release referencing that digest.
3. Deploy to development and run smoke tests before promotion.
4. Configure a multi-phase production canary with meaningful bake time and a manual approval before full rollout.
5. Introduce a controlled error in the canary and demonstrate that verification or operator review stops progression.
6. Roll back or abandon the faulty rollout, then create a corrected release without rebuilding the prior good artifact.
7. Export rollout events, approvals, metrics, and incident notes as evidence.

Validation Evidence

- The same digest is promoted across environments.
- A failing canary does not receive full traffic.
- Operators can pause and recover without project-wide administrator access.
- Release evidence identifies who, what, where, when, and health outcome.

Cleanup

- Delete training delivery pipelines, targets, releases where supported, services, images, and monitoring artifacts.

Interview Questions and Model Answers

1. Why separate build from deployment?

Artifact creation and production authorization are different trust decisions and should use different identities, evidence, and owners.

2. What makes a canary meaningful?

Representative exposure, adequate bake time, sensitive service indicators, segmentation, and a tested stop or rollback path.

3. Why reuse the same digest?

Rebuilding creates different bytes and invalidates evidence collected in earlier environments.

4. When is roll-forward safer than rollback?

When data or schema changes are not backward compatible and a small corrective change is less risky than returning to the old version.

Chapter Summary

- Cloud Deploy organizes controlled promotion of immutable releases.
- Progressive delivery limits blast radius only when signals and exposure are representative.
- Release verification must reflect user outcomes and service objectives.
- Rollback, roll-forward, and break-glass procedures require pre-production testing.

Official Google Cloud References

<https://docs.cloud.google.com/deploy/docs>

<https://docs.cloud.google.com/deploy/docs/deployment-strategies/canary>

<https://docs.cloud.google.com/deploy/docs/roll-back-target>

Chapter 45 - CI/CD Security, Provenance, and Policy Enforcement

Keyless federation, least privilege, attestations, Binary Authorization, policy gates, and incident response

Learning Objectives

- Authenticate external pipelines with Workload Identity Federation and short-lived credentials.
- Build a chain of evidence from source revision through artifact digest, provenance, approval, and deployment.
- Use Binary Authorization and policy controls to reject untrusted artifacts.
- Respond to credential, builder, dependency, and artifact compromise.

Why This Matters in Production

CI/CD Security, Provenance, and Policy Enforcement is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

45.1 Keyless Pipeline Authentication

External CI systems should exchange their native OIDC or cloud identity for short-lived Google Cloud credentials through Workload Identity Federation. Attribute mappings and conditions restrict which repository, branch, environment, organization, or workflow can impersonate a service account. Avoid a single federation rule that accepts any repository in a provider. Production deployment identities should require both trusted federation attributes and target-specific IAM.

45.2 Provenance and Attestation

Provenance describes how an artifact was produced: source, builder, parameters, dependencies, and output digest. An attestation is a signed statement about an artifact, such as passing security review. These are not interchangeable. A policy may require provenance from an approved builder plus independent attestations for vulnerability, license, or production approval. Evidence must be bound to immutable digests.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

45.3 Binary Authorization and Enforcement

Binary Authorization evaluates policy before supported workloads run an image. Use it with Artifact Registry, vulnerability data, build provenance, and attestations. Begin with monitoring or non-production enforcement, measure denied workloads, correct gaps, and then expand. Break-glass should be rare, time-bound, logged, alerted, and reviewed. It must not become a permanent deployment path.

45.4 Supply-Chain Incident Response

When a source credential, builder, dependency, signing key, or image is compromised, identify affected digests, freeze promotion, revoke federation bindings or identities, invalidate attestations, block deployment, rotate secrets, rebuild from trusted inputs, and search runtime inventory. The incident record should distinguish exposure from actual execution and preserve evidence needed for legal or compliance review.

Production Design Checklist

- Authenticate external pipelines with Workload Identity Federation and short-lived credentials.
- Build a chain of evidence from source revision through artifact digest, provenance, approval, and deployment.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.

- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Federation token denied	Issuer, audience, attribute mapping, condition, clock, or service-account binding mismatch	Inspect token claims and federation audit data; correct narrow condition rather than removing it
Trusted build denied at deploy	Attestation attached to tag or wrong digest, wrong project, or policy scope	Resolve deployed digest, list attestations, and verify policy evaluation context
Untrusted image allowed	Policy not enforced on target or break-glass path overused	Verify target integration, policy scope, admission logs, and alert on emergency bypass
Incident rebuild produces same vulnerable result	Compromised dependency or builder remains in trusted inputs	Rotate trust roots, pin verified dependencies, use clean builder, and compare provenance

Hands-on Lab 45: Create a Keyless, Policy-Gated Deployment

Prerequisites

- An external or simulated OIDC pipeline, Workload Identity Federation, Artifact Registry, Cloud Build or trusted builder, and a supported deployment target.

Implementation

1. Create a workload identity pool/provider with conditions limited to one repository and protected production environment.
2. Allow the federated principal to impersonate a narrowly scoped build or deploy service account.
3. Build an artifact and capture provenance and immutable digest.
4. Create an approval attestation tied to the digest.
5. Configure Binary Authorization or an equivalent deploy gate to require the approved evidence.
6. Attempt deployment from an unapproved branch and with an unattested digest; record both denials.
7. Deploy the approved digest, then run a tabletop compromise response for the federation binding and signer.

Validation Evidence

- No static credential is stored in the CI platform.
- Repository and environment claims are enforced.
- Unattested or untrusted digests are denied.
- Revocation and recovery actions are documented and tested.

Cleanup

- Remove test providers, bindings, attestations, policies, workloads, images, and service accounts.

Interview Questions and Model Answers

1. What is the benefit of Workload Identity Federation?

It exchanges external identity for short-lived Google Cloud credentials and avoids distributing service-account keys.

2. Is provenance the same as approval?

No. Provenance describes how an artifact was built; approval is a separate trust decision that can be expressed as an attestation.

3. What is Binary Authorization?

A deploy-time policy control that evaluates artifact evidence and rules before allowing supported workloads to run.

4. What should break-glass do?

Provide a narrowly scoped, time-bound, logged emergency path with alerts and mandatory post-use review.

Chapter Summary

- Keyless federation reduces credential exposure but depends on precise claim conditions.
- Artifact trust is a chain from source and builder to digest, evidence, policy, and runtime.
- Policy enforcement should be staged and observable before broad production rollout.
- Supply-chain incidents require inventory, revocation, rebuild, and runtime search.

Official Google Cloud References

<https://docs.cloud.google.com/iam/docs/workload-identity-federation-with-deployment-pipelines>

<https://docs.cloud.google.com/binary-authorization/docs>

<https://docs.cloud.google.com/build/docs/securing-builds/store-manage-build-artifacts>

Chapter 46 - Cloud Monitoring, Managed Prometheus, SLOs, and Alerting

Metrics, dashboards, service objectives, error budgets, alert quality, and operations at scale

Learning Objectives

- Design metric scopes, monitored resources, labels, dashboards, and retention.
- Collect application and Kubernetes metrics with Managed Service for Prometheus and OpenTelemetry.
- Define SLIs, SLOs, and error budgets that represent user experience.
- Create actionable alerts with routing, suppression, runbooks, and quality review.

Why This Matters in Production

Cloud Monitoring, Managed Prometheus, SLOs, and Alerting is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

46.1 Monitoring Architecture

Cloud Monitoring automatically collects many Google Cloud service metrics and can receive custom, Prometheus, and OpenTelemetry metrics. A metric scope can centralize visibility across projects, but access and cost boundaries must remain explicit. Labels enable segmentation but high-cardinality labels can create cost and query problems. Dashboards should be organized around services and user journeys, not around a random list of infrastructure charts.

46.2 Managed Service for Prometheus

Managed Service for Prometheus supports PromQL-based monitoring for Kubernetes and other workloads. Collection choices include managed collection and OpenTelemetry collectors. Standardize metric names, units, label sets, scrape intervals, and ownership. Protect endpoints and avoid placing customer identifiers, request IDs, or unbounded values in labels.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

46.3 SLIs, SLOs, and Error Budgets

An SLI is a measured indicator such as successful request ratio, latency under a threshold, freshness, or job completion. An SLO is a target over a period. Error budget is the allowed unreliability. Choose indicators users experience, define valid events and exclusions, and document data gaps. Burn-rate alerting detects both fast and slow exhaustion more reliably than a single static threshold.

46.4 Alert Engineering

An alert must identify a condition requiring human or automated action. Include affected service, scope, severity, current value, likely causes, owner, dashboard, and runbook. Route by ownership and urgency, group duplicates, suppress expected maintenance, and avoid paging on symptoms that self-heal within seconds. Review false positives, missed incidents, acknowledgment time, and time to useful action.

Production Design Checklist

- Design metric scopes, monitored resources, labels, dashboards, and retention.
- Collect application and Kubernetes metrics with Managed Service for Prometheus and OpenTelemetry.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.

- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Metric absent	API/agent/collector disabled, label mismatch, permission, or delayed ingestion	Check collector health, monitored resource, metric descriptor, IAM, and time range
Prometheus cost grows	Excess series cardinality or aggressive scrape interval	Identify high-cardinality labels, drop or aggregate them, and adjust collection frequency
SLO shows healthy during incident	SLI excludes failing path or denominator is wrong	Compare with user reports and raw events; redefine valid/total events and segmentation
Alert storm	Multiple dependent symptoms and no grouping or inhibition	Page on service impact, group by incident, route diagnostics as non-paging notifications

Hands-on Lab 46: Implement an SLO and Multi-Window Burn-Rate Alert

Prerequisites

- A small Cloud Run or GKE service, Cloud Monitoring, telemetry, and a notification channel.

Implementation

1. Instrument request count, error count, and latency using supported metrics or OpenTelemetry.
2. Create a service dashboard with traffic, errors, latency, saturation, and deployment markers.
3. Define a user-focused availability SLI and a latency SLI, including exclusions and measurement window.
4. Set SLO targets and calculate the error budget.
5. Create fast-burn and slow-burn alerts with different windows and severity.
6. Inject controlled failures and confirm the correct alert and dashboard context.
7. Review series cardinality, notification routing, and runbook quality.

Validation Evidence

- SLI math matches raw request evidence.
- Burn-rate alerts detect controlled failure without excessive delay.
- Notifications identify service, scope, owner, dashboard, and runbook.
- High-cardinality data is not exported as metric labels.

Cleanup

- Remove test alerts, notification channels, dashboards, synthetic load, and temporary resources.

Interview Questions and Model Answers

1. What is the difference between an SLI and SLO?

An SLI is the measured reliability indicator; an SLO is the target for that indicator over a defined period.

2. Why use burn-rate alerts?

They indicate how quickly the error budget is being consumed and can detect both severe short incidents and slower persistent degradation.

3. Why are high-cardinality labels dangerous?

They create many time series, increasing cost and query complexity and potentially degrading monitoring usefulness.

4. What makes an alert actionable?

It represents material impact and includes enough ownership, context, and recovery guidance for a responder to act.

Chapter Summary

- Monitoring architecture includes collection, scope, labels, ownership, cost, and retention.
- Managed Prometheus and OpenTelemetry support portable telemetry when conventions are controlled.
- SLOs translate reliability into measurable user outcomes and error budgets.
- Alert quality is an engineering lifecycle, not a one-time threshold configuration.

Official Google Cloud References

<https://docs.cloud.google.com/monitoring/docs/monitoring-overview>

<https://docs.cloud.google.com/stackdriver/docs/managed-prometheus>

<https://docs.cloud.google.com/stackdriver/docs/solutions/slo-monitoring>

Chapter 47 - Cloud Logging, OpenTelemetry, Trace, and Application Monitoring

Structured logs, telemetry pipelines, distributed context, sampling, retention, and incident analysis

Learning Objectives

- Design structured logging, log routing, storage, retention, and access controls.
- Instrument metrics, traces, and logs using OpenTelemetry and the Google-built collector where appropriate.
- Correlate requests across services with trace and resource context.
- Control telemetry cost, privacy, sampling, and failure modes.

Why This Matters in Production

Cloud Logging, OpenTelemetry, Trace, and Application Monitoring is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

47.1 Structured Logging and Routing

Logs should be machine-parseable events with severity, service, environment, version, trace context, event type, and safe diagnostic fields. Do not log secrets, tokens, raw personal data, or full payloads by default. Log Router sinks can direct selected logs to buckets, BigQuery, Pub/Sub, or other projects. Exclusions reduce ingestion cost but can also remove forensic evidence, so every exclusion needs an owner and risk review.

47.2 OpenTelemetry Pipeline

OpenTelemetry provides common APIs, SDKs, semantic conventions, and collectors for telemetry. Google Cloud supports OTLP-based ingestion paths and a Google-built OpenTelemetry Collector. Treat the collector as production infrastructure: monitor queue, export errors, dropped telemetry, memory, configuration version, and endpoint authentication. Use resource attributes to identify service and environment consistently.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

47.3 Trace Correlation and Sampling

Distributed traces explain how a request crosses services and where latency or errors occur. Propagate trace context through HTTP, messaging, and background work. Head sampling decides early and is inexpensive; tail sampling can retain unusual traces but requires more collector capacity. Keep error and high-latency traces at higher rates, while controlling normal-traffic volume.

47.4 Retention, Privacy, and Telemetry Reliability

Different data classes need different retention and access. Security audit, application diagnostic, and high-volume debug logs should not share one uncontrolled bucket. Define data residency, CMEK where required, retention locks carefully, and access logging. During a telemetry outage, applications should continue serving where safe; collectors need bounded buffers and must not exhaust workload memory or disk.

Production Design Checklist

- Design structured logging, log routing, storage, retention, and access controls.
- Instrument metrics, traces, and logs using OpenTelemetry and the Google-built collector where appropriate.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.

- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Logs missing from central project	Sink filter, writer identity, destination IAM, exclusion, or scope	Inspect sink, writer permission, destination, exclusions, and source log entry
Trace is fragmented	Context not propagated through async or proxy boundary	Inspect headers/attributes; instrument messaging producer and consumer; standardize propagator
Collector drops data	Backpressure, export failure, memory limit, or queue too small	Monitor collector metrics, tune queues/batches, scale, and fix destination auth/network
Logging bill spikes	Debug flood, high-volume payloads, broad retention, or duplicate export	Find top log names/labels, reduce at source, sample, narrow sinks, and adjust retention

Hands-on Lab 47: Build a Correlated Telemetry Pipeline

Prerequisites

- A two-service application, OpenTelemetry SDK or agent, Cloud Logging, Monitoring, and Trace.

Implementation

1. Emit structured application logs with service, version, severity, event type, and trace context.
2. Instrument incoming and outgoing calls with OpenTelemetry traces and selected metrics.
3. Deploy a collector or use direct supported export, documenting the reliability trade-off.
4. Create a log sink that routes a narrow security or audit subset to a central project.
5. Configure sampling that retains errors and slow requests while limiting normal traffic.
6. Generate one successful and one failing request and follow each through logs, metrics, and traces.
7. Simulate collector export failure and confirm bounded buffering and recovery.

Validation Evidence

- A request can be correlated across two services.
- Sensitive values are absent from telemetry.
- Sink destination receives only intended log classes.
- Collector failure does not exhaust application resources.

Cleanup

- Delete test sinks, buckets, datasets, collectors, dashboards, and applications after exporting sanitized evidence.

Interview Questions and Model Answers

1. Why use structured logs?

They allow reliable filtering, aggregation, routing, correlation, and automated analysis without parsing free-form text.

2. What is the role of an OpenTelemetry Collector?

It receives, processes, samples, batches, and exports telemetry, decoupling application instrumentation from backends.

3. What is tail sampling?

A sampling decision made after observing more of a trace, allowing retention of errors or slow traces at the cost of collector resources.

4. Why can log exclusions be risky?

They can permanently remove data needed for operations, security investigation, or compliance if applied too broadly.

Chapter Summary

- Telemetry must be structured, correlated, privacy-aware, and cost-controlled.
- OpenTelemetry creates portability but collectors require capacity and reliability engineering.
- Sampling is a risk and cost decision, not just a percentage.
- Log routing, retention, and exclusions are part of the evidence architecture.

Official Google Cloud References

<https://docs.cloud.google.com/logging/docs>

<https://docs.cloud.google.com/stackdriver/docs/instrumentation/google-built-otel>

<https://docs.cloud.google.com/trace/docs/overview>

Chapter 48 - Cloud Audit Logs, Asset Inventory, Policy Intelligence, and Fleet Operations

Evidence, inventory, access analysis, configuration history, automation, and operational control

Learning Objectives

- Differentiate Admin Activity, Data Access, System Event, and Policy Denied audit logs.
- Use Cloud Asset Inventory to search, export, monitor, and analyze resource metadata.
- Apply Policy Intelligence to analyze, troubleshoot, simulate, and reduce access.
- Create controlled fleet operations with inventory, change records, and reversible automation.

Why This Matters in Production

Cloud Audit Logs, Asset Inventory, Policy Intelligence, and Fleet Operations is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

48.1 Audit Evidence Architecture

Cloud Audit Logs record administrative and data activity with different availability, volume, and configuration characteristics. Centralize required logs in a protected project, restrict deletion, and define retention based on investigation and compliance needs. Data Access logs can be high volume and may require deliberate enablement and filtering. Policy Denied records explain authorization failures and are valuable for both troubleshooting and threat detection.

48.2 Cloud Asset Inventory

Cloud Asset Inventory provides a global inventory and time-based metadata history for supported Google Cloud resources. Use search for discovery, export snapshots to BigQuery or Cloud Storage for analysis, and feed changes to Pub/Sub for event-driven controls. Inventory is eventually consistent; critical operations should confirm state through the service API when immediate accuracy is required.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

48.3 Policy Intelligence

Policy Analyzer answers who can access what. Policy Troubleshooter explains why a principal is allowed or denied. Recommender surfaces opportunities to reduce excess permission, and Policy Simulator predicts the effect of IAM or organization-policy changes. Recommendations are evidence, not automatic truth; review workload behavior, emergency access, inherited bindings, and business ownership before applying.

48.4 Fleet Operations and Change Control

Fleet operations should select resources by explicit labels, folders, projects, or inventory queries; use dry-run; cap concurrency; record before-and-after state; and stop on error thresholds. Automation must distinguish unreachable from noncompliant resources. High-risk changes such as IAM, firewall, key, logging, or organization policy require canary scope and tested rollback.

Production Design Checklist

- Differentiate Admin Activity, Data Access, System Event, and Policy Denied audit logs.
- Use Cloud Asset Inventory to search, export, monitor, and analyze resource metadata.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.

- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Expected audit event absent	Log class disabled, excluded, wrong scope, or event not audited	Verify audit configuration, log name, resource scope, exclusions, and service documentation
Asset search misses recent change	Eventual consistency or unsupported field/type	Wait bounded interval, query service API, and check supported asset type
Recommender removal breaks workload	Unused observation window missed periodic access	Use simulator and staged removal; monitor denial logs; retain recovery path
Fleet job changes wrong resources	Selector too broad or labels inconsistent	Require preview list, ownership validation, canary batch, and concurrency cap

Hands-on Lab 48: Create a Central Audit and Asset-Change Control

Prerequisites

- A sandbox organization or folder, central logging project, Cloud Asset Inventory, Pub/Sub, and BigQuery.

Implementation

1. Create an organization or folder log sink for selected audit log classes into a protected central project.
2. Verify sink writer permissions and restrict access to log buckets.
3. Export a Cloud Asset Inventory snapshot to BigQuery and query resources missing owner labels.
4. Create an asset feed for a narrow resource type and publish changes to Pub/Sub.
5. Build a small automation that previews but does not automatically modify noncompliant resources.
6. Use Policy Analyzer and Troubleshooter for one IAM path, then simulate a proposed role reduction.
7. Perform a canary remediation and verify audit, inventory, and rollback evidence.

Validation Evidence

- Central logs are immutable to workload administrators within the lab model.
- Asset snapshot and event feed identify expected resources and changes.
- Policy analysis explains effective inherited access.
- Fleet remediation is previewed, canaried, and reversible.

Cleanup

- Delete test sinks, datasets, feeds, topics, automation identities, and sample resources.

Interview Questions and Model Answers

1. What is the difference between audit logs and asset inventory?

Audit logs record actions and decisions; asset inventory describes resource metadata and its changes over time.

2. Why is Cloud Asset Inventory not always suitable for immediate enforcement?

It is eventually consistent, so a service API or transaction-specific control may be needed for real-time decisions.

3. What does Policy Troubleshooter answer?

Why a principal is allowed or denied a permission on a resource in a particular context.

4. What makes fleet automation safe?

Explicit selection, preview, canary scope, concurrency limits, stop conditions, before-and-after evidence, and rollback.

Chapter Summary

- Audit logs, inventory, and policy analysis answer different evidence questions.
- Central log architecture must protect records from workload administrators.
- Recommendations should be simulated and staged before broad access reduction.
- Fleet operations need the same safety controls as software releases.

Official Google Cloud References

<https://docs.cloud.google.com/logging/docs/audit>

<https://docs.cloud.google.com/asset-inventory/docs/asset-inventory-overview>

<https://docs.cloud.google.com/policy-intelligence/docs/overview>

Chapter 49 - Secret Manager, Cloud KMS, HSM, EKM, and Certificate Lifecycle

Secrets, cryptographic keys, customer control, rotation, availability, and recovery

Learning Objectives

- Differentiate secrets, encryption keys, signing keys, certificates, and identity credentials.
- Design Secret Manager replication, access, versioning, rotation, and application retrieval.
- Choose Cloud KMS, Cloud HSM, Cloud EKM, and CMEK patterns based on risk and availability.
- Operate key and certificate lifecycle without creating unrecoverable outages.

Why This Matters in Production

Secret Manager, Cloud KMS, HSM, EKM, and Certificate Lifecycle is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

49.1 Secrets and Runtime Access

Secret Manager stores sensitive values as versioned secrets. Applications should retrieve secrets using attached or federated identities, cache them only as long as justified, and avoid environment-variable exposure where process inspection or crash dumps are a concern. Use separate secrets by environment and ownership. Replication and regional service choices should follow residency and availability requirements.

49.2 Key Hierarchy and Protection Levels

Cloud KMS organizes key rings, keys, and key versions in locations. Software keys, HSM-backed keys, single-tenant HSM options, and externally managed keys provide different control, latency, availability, cost, and operational obligations. CMEK gives customers control over keys protecting supported service data. Cloud KMS Autokey can automate CMEK provisioning for supported resources while preserving centralized key governance.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

49.3 Rotation, Destruction, and Separation of Duties

Secret rotation changes the value and coordinates consumers. Key rotation creates a new primary version for future operations while old versions may still be required for decryption. Disable before destroy, use scheduled destruction windows, and test recovery. Separate key administration, key usage, application deployment, and audit roles. An administrator who can destroy keys should not be the only person able to approve recovery.

49.4 Certificates and Availability

Certificates bind names or identities to keys and require issuance, renewal, deployment, revocation, and monitoring. Inventory certificate expiry and chain validity, automate renewal, and alert before deadlines. External key management creates a runtime dependency on the external system and network; design failover, latency, maintenance, and incident procedures before choosing it for high-availability services.

Production Design Checklist

- Differentiate secrets, encryption keys, signing keys, certificates, and identity credentials.
- Design Secret Manager replication, access, versioning, rotation, and application retrieval.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.

- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Application sees old secret	Cache, pinned version, replication delay, or failed rollout	Inspect requested version, cache TTL, deployment state, and secret access logs
CMEK resource creation denied	KMS location, IAM, service agent, org policy, or key disabled	Verify location compatibility, service-agent role, key state, and effective policy
Rotation causes outage	Consumers not compatible with overlap or secret changed atomically	Use dual-value overlap, staged rollout, readiness check, and rollback version
Key destruction threatens data	Retention dependency not inventoried	Stop destruction, restore scheduled version where possible, and map encrypted assets before resuming

Hands-on Lab 49: Operate a Secret and CMEK Rotation Safely

Prerequisites

- A sandbox project, Secret Manager, Cloud KMS, a low-cost application, and a supported CMEK-enabled resource.

Implementation

1. Create separate runtime and administration identities with least privilege.
2. Store a test credential as a secret and deploy an application that accesses a specific version through its identity.
3. Implement overlapping secret versions and staged consumer rotation.
4. Create a key ring and symmetric encryption key in a compatible location; enable a test resource with CMEK.
5. Rotate the key and prove old ciphertext remains decryptable while new operations use the new primary version.
6. Disable a non-primary test key version and demonstrate recovery before any destruction.
7. Document certificate or secret expiry monitoring and an emergency access procedure.

Validation Evidence

- Secret values never appear in source, image, Terraform state, or logs.
- Runtime identity can access only the required secret and key operation.
- Rotation has no service interruption.
- Disable, recovery, and destruction approvals are evidenced.

Cleanup

- Delete test secrets, resources, keys only when safe and permitted, identities, and monitoring artifacts.

Interview Questions and Model Answers

1. What is the difference between secret rotation and key rotation?

Secret rotation changes a credential value and consumers; key rotation creates a new key version for future cryptographic operations while retaining older versions as needed.

2. Why is key location important?

Key and protected resource compatibility, residency, latency, and availability can depend on location.

3. What is CMEK?

Customer-managed encryption keys in Cloud KMS used by supported Google Cloud services to protect data at rest under customer-controlled lifecycle and IAM.

4. Why can external keys reduce availability?

Google Cloud cryptographic operations depend on the external key manager and its network, latency, maintenance, and authorization behavior.

Chapter Summary

- Secrets, keys, and certificates have different lifecycle and access semantics.
- Rotation must be designed with compatibility overlap and rollback.
- CMEK increases customer control and operational responsibility.
- Destruction and external-key dependencies require explicit recovery engineering.

Official Google Cloud References

<https://docs.cloud.google.com/secret-manager/docs/overview>

<https://docs.cloud.google.com/kms/docs/key-management-service>

<https://docs.cloud.google.com/kms/docs/cmek>

Chapter 50 - Security Command Center and Cloud Security Posture Management

Asset risk, misconfiguration, vulnerability, threat findings, attack paths, ownership, and remediation

Learning Objectives

- Explain Security Command Center activation scope, findings, sources, and posture workflows.
- Prioritize risk using exposure, exploitability, criticality, path, and business context.
- Route findings to accountable owners with SLAs, exceptions, and verification.
- Integrate posture management into platform templates and change prevention.

Why This Matters in Production

Security Command Center and Cloud Security Posture Management is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

50.1 Security Command Center Operating Model

Security Command Center centralizes security posture, vulnerability, threat, and asset-risk information. Organization-level activation generally provides broader visibility than isolated project activation. Available capabilities depend on service tier and configuration. Define who owns platform findings, workload findings, triage, remediation, risk acceptance, and executive reporting before turning on a large volume of detections.

50.2 Finding Triage and Risk Prioritization

Severity alone is insufficient. Prioritize by internet exposure, sensitive data, privilege, exploitability, active threat evidence, attack path, affected business service, and compensating controls. Group related findings by root cause, such as one overly broad service account used across many resources. Suppress only with documented rationale, expiration, owner, and alternative control.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

50.3 Remediation Workflow

A finding lifecycle includes ingestion, enrichment, ownership, SLA, remediation, verification, and closure. Automate low-risk corrections only after canary testing. High-risk changes such as disabling identities, changing routes, or enforcing encryption require application-owner coordination. Verify closure from the source control and from the actual resource, because a finding status alone may not prove the risk is removed.

50.4 Shift-Left Posture

Convert recurring findings into Terraform module defaults, organization policies, CI checks, deployment policies, and platform documentation. Measure time to owner, time to remediate, reopened findings, accepted-risk age, coverage gaps, and repeated root causes. The target is not zero findings; it is rapid control of material risk and reduction of preventable recurrence.

Production Design Checklist

- Explain Security Command Center activation scope, findings, sources, and posture workflows.
- Prioritize risk using exposure, exploitability, criticality, path, and business context.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.

- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Finding has no owner	Missing resource labels, service mapping, or routing rules	Enforce ownership metadata and map folders/projects to accountable teams
Thousands of duplicate findings	One shared root cause or repeated scanner event	Group by resource/control/root cause and remediate platform source
Finding closed but risk remains	Manual status change or delayed rescan	Validate resource configuration and wait for authoritative detector confirmation
Automation breaks workload	Remediation lacked canary or context	Rollback, suspend automation, add preconditions and application-owner approval

Hands-on Lab 50: Build a Finding-to-Remediation Workflow

Prerequisites

- A sandbox organization/project with Security Command Center available, test resources, Pub/Sub or export destination, and an issue workflow.

Implementation

1. Activate the appropriate lab scope and inventory enabled detection sources.
2. Create two deliberate low-risk misconfigurations in a sandbox resource.
3. Export or route findings to a central queue or analysis destination.
4. Enrich each finding with owner, environment, criticality, exposure, and service context.
5. Define remediation SLA and exception fields with expiry.
6. Fix one issue manually and one through canaried automation; verify detector closure.
7. Create a dashboard for open risk, age, ownership gaps, recurrence, and accepted exceptions.

Validation Evidence

- Every finding has an accountable owner and due date.
- Risk priority uses context beyond severity.
- Closure is verified against actual resource state.
- Automation can be paused and rolled back.

Cleanup

- Remove deliberate misconfigurations, test exports, topics, automation, and dashboards.

Interview Questions and Model Answers

1. What is CSPM?

Cloud security posture management continuously identifies and helps manage misconfiguration, exposure, compliance, and other cloud-control risks.

2. Why not prioritize only by severity?

Severity lacks business criticality, exposure, exploitability, attack path, active threat, and compensating-control context.

3. What is a good exception?

A documented, approved, time-bound risk acceptance with owner, rationale, compensating controls, and review date.

4. How do findings improve platform engineering?

Repeated root causes are converted into secure defaults, policy, tests, and golden-path controls so they do not recur.

Chapter Summary

- Security Command Center requires an operating model, not only activation.
- Risk prioritization combines technical and business context.
- Closure must be verified from authoritative resource state.
- The strongest remediation prevents the same control failure in future deployments.

Official Google Cloud References

<https://docs.cloud.google.com/security-command-center/docs/security-command-center-overview>

<https://docs.cloud.google.com/security-command-center/docs/optimize-security-command-center>

<https://docs.cloud.google.com/security-command-center/docs/how-to-investigate-threats>

Chapter 51 - Google Security Operations and Detection Engineering

Telemetry ingestion, normalized data, detections, investigations, SOAR, playbooks, and incident learning

Learning Objectives

- Explain Google Security Operations SIEM and SOAR capabilities and data flow.
- Design ingestion, parsing, normalization, retention, access, and detection ownership.
- Write, test, tune, and govern detection rules with measurable coverage.
- Operate investigation, containment, playbooks, evidence, and post-incident improvement.

Why This Matters in Production

Google Security Operations and Detection Engineering is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

51.1 Security Telemetry Architecture

Google Security Operations centralizes and analyzes security and network telemetry for detection, investigation, and response. Define authoritative sources, transport, parser, normalization to the Unified Data Model, timestamp quality, tenant or environment labels, retention, and data residency. Ingestion health must be monitored because a silent source failure creates a detection blind spot.

51.2 Detection Engineering

A detection has a hypothesis, required telemetry, logic, severity, expected false positives, response owner, and test cases. Version rules, peer review them, test against historical and synthetic events, and measure precision, recall proxies, alert volume, and time to triage. Map detections to threat behaviors, but do not treat framework coverage as proof that the implementation works.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

51.3 Investigation and Response

Investigators pivot across users, assets, domains, processes, network events, cloud audit data, and threat intelligence. Preserve a timeline and evidence references. Containment actions such as disabling accounts, revoking tokens, isolating workloads, blocking indicators, or rotating secrets must be scoped and reversible. Coordinate with service owners so containment does not destroy evidence or cause uncontrolled outages.

51.4 SOAR and Playbooks

SOAR playbooks automate enrichment, ticketing, notification, containment, and evidence collection. Use human approval for destructive or broad actions until confidence is high. Every integration credential is privileged and needs rotation, least privilege, monitoring, and failure handling. Post-incident work updates detections, parsing, runbooks, platform controls, training, and recovery tests.

Production Design Checklist

- Explain Google Security Operations SIEM and SOAR capabilities and data flow.
- Design ingestion, parsing, normalization, retention, access, and detection ownership.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Source stops ingesting	Connector, credential, quota, network, parser, or schema change	Alert on freshness by source; inspect connector and parser; backfill if supported
Rule alerts on benign admin work	Context missing or maintenance workflow not modeled	Add identity, asset, change-ticket, and time context; use bounded suppression
Playbook containment fails	Integration role, API quota, target state, or approval timeout	Log every step, implement retry and fallback, and escalate to manual runbook
Investigation timeline conflicts	Clock skew, delayed ingestion, duplicated events, or timezone confusion	Normalize timestamps, retain source time and ingest time, and document uncertainty

Hands-on Lab 51: Create and Test a Cloud Threat Detection

Prerequisites

- A Google Security Operations lab or conceptual sandbox, sample audit/network events, detection rule authoring access, and a case workflow.

Implementation

1. Define a detection hypothesis for suspicious service-account privilege use or impossible administrative behavior.
2. List required log sources, normalized fields, exclusions, and expected benign cases.
3. Ingest or replay sanitized sample events and verify parsing into the expected data model.
4. Write a versioned detection rule and unit test it with positive and negative samples.
5. Generate one controlled matching event and create a case with asset, identity, and timeline enrichment.
6. Run a playbook that gathers context and requests human approval before a reversible containment action.
7. Close the case with root cause, control improvements, rule tuning, and evidence references.

Validation Evidence

- Source freshness and parser health are visible.
- The rule detects the positive sample and ignores known negatives.
- The case records a reproducible timeline.
- Containment requires controlled authorization and can be reversed.

Cleanup

- Remove test rules, samples, cases, integrations, and temporary credentials according to lab policy.

Interview Questions and Model Answers

1. What is the difference between SIEM and SOAR?

SIEM ingests, normalizes, searches, and detects across security telemetry; SOAR orchestrates investigation and response workflows.

2. What makes a detection production-ready?

A clear hypothesis, required data, version control, tests, ownership, severity, tuning, response procedure, and coverage monitoring.

3. Why monitor source freshness?

A working rule cannot detect events from a source that silently stopped ingesting.

4. When should containment be automated?

When scope, confidence, reversibility, authorization, and failure handling are proven; destructive actions often retain human approval.

Chapter Summary

- Security operations depends on reliable, normalized telemetry.
- Detection rules are software products with tests, versions, owners, and performance metrics.
- Investigations need timelines, evidence, business context, and reversible containment.
- Post-incident improvements must change controls, detections, and recovery readiness.

Official Google Cloud References

<https://docs.cloud.google.com/chronicle/docs>

<https://docs.cloud.google.com/chronicle/docs/overview>

<https://docs.cloud.google.com/chronicle/docs/secops/understand-the-secops-platform>

Chapter 52 - VPC Service Controls, Cloud Armor, Firewall Policies, and Zero Trust

Data perimeters, context-aware access, edge protection, network policy, and controlled exceptions

Learning Objectives

- Explain VPC Service Controls service perimeters, access levels, ingress, egress, and dry-run.
- Design Cloud Armor protections for internet-facing services.
- Use hierarchical firewall policies and secure administration paths.
- Implement Zero Trust decisions using identity, device, context, service, and data sensitivity.

Why This Matters in Production

VPC Service Controls, Cloud Armor, Firewall Policies, and Zero Trust is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

52.1 VPC Service Controls

VPC Service Controls mitigates data exfiltration by placing supported Google-managed services inside service perimeters and controlling ingress and egress. It complements IAM; it does not replace identity permissions. Start with dry-run, inventory supported services and workflows, review violations, design access levels, and then enforce incrementally. Perimeters can break automation, consoles, cross-project services, or external tools if dependencies are not mapped.

52.2 Cloud Armor and Edge Protection

Cloud Armor applies web application firewall and DDoS-related policies to supported load-balanced services. Use preconfigured rules, custom expressions, rate limiting, threat intelligence where available, logging, and preview mode. Tune rules using representative traffic and application-owner review. Edge controls reduce exposure but do not correct vulnerable application logic or authorization.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

52.3 Firewall Policy and Administration

Hierarchical firewall policies provide centrally governed rules across folders and projects, while VPC firewall rules address network-specific needs. Prefer identity-aware administration through IAP, OS Login, bastions only when necessary, and private access. Deny rules and broad egress restrictions require dependency inventories and emergency access. Log high-risk rule matches without enabling unlimited noisy logging.

52.4 Zero Trust and Exceptions

Zero Trust evaluates each access request from identity, device, network, workload, service, and resource context rather than trusting location alone. Exceptions must be narrow, monitored, expiring, and owned. Test failures: DNS to restricted Google APIs, service-agent calls, build systems, external integrations, and disaster-recovery paths are common perimeter blind spots.

Production Design Checklist

- Explain VPC Service Controls service perimeters, access levels, ingress, egress, and dry-run.
- Design Cloud Armor protections for internet-facing services.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.

- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Perimeter blocks valid pipeline	Missing ingress/egress rule, access level, unsupported path, or wrong project placement	Use dry-run logs and violation analysis; create narrow rule tied to identity and service
Cloud Armor blocks real users	Rule sensitivity, proxy/header assumption, or rate limit too low	Use preview, sample logs, segment clients, and tune with application evidence
Firewall deny causes outage	Dependency or service-agent traffic not inventoried	Rollback canary policy, inspect flow/audit logs, document dependency, then redesign
Restricted API DNS fails	Private Google Access or restricted.googleapis.com DNS/routing error	Verify subnet setting, DNS policy, routes, and perimeter-supported endpoint

Hands-on Lab 52: Protect a Data Service with a Dry-Run Perimeter and Edge Policy

Prerequisites

- Two sandbox projects, a supported data service, VPC Service Controls permissions, a small load-balanced service, and Cloud Armor.

Implementation

1. Inventory the data service, calling identities, projects, networks, and external tools.
2. Create a service perimeter in dry-run mode and generate allowed and disallowed access attempts.
3. Analyze violations and design narrow ingress/egress policies and access levels.
4. Enforce the perimeter for the lab project and verify restricted Google API access.
5. Create a Cloud Armor policy in preview mode with one preconfigured rule and one rate limit.
6. Replay benign and malicious test traffic, tune the policy, and enforce only after review.
7. Document emergency access, rollback, logging, and exception expiration.

Validation Evidence

- IAM allow alone cannot bypass the enforced perimeter.
- Approved identities and paths continue to work.
- Cloud Armor logs show preview and enforcement decisions.
- Exceptions are narrow, owned, monitored, and time-bound.

Cleanup

- Remove perimeter, access policies, edge policies, test services, projects, and logs according to retention needs.

Interview Questions and Model Answers

1. Does VPC Service Controls replace IAM?

No. IAM decides who can perform actions; service perimeters add contextual boundaries to reduce data exfiltration.

2. Why start with dry-run?

It reveals legitimate workflows that enforcement would block without exposing protected data through an untested policy.

3. What does Cloud Armor protect?

Supported internet-facing services from classes of malicious traffic and abusive patterns at the edge; it is not a substitute for secure application code.

4. What is a good exception?

The smallest identity, service, direction, resource, and time scope that supports a justified workflow, with monitoring and expiration.

Chapter Summary

- Data perimeters, identity policy, and network controls are complementary.
- Dry-run and preview modes are essential for safe enforcement.
- Edge protection requires tuning against real application behavior.
- Zero Trust treats every exception as a managed risk object.

Official Google Cloud References

<https://docs.cloud.google.com/vpc-service-controls/docs/overview>

<https://docs.cloud.google.com/armor/docs/cloud-armor-overview>

<https://docs.cloud.google.com/firewall/docs/firewall-policies-overview>

Chapter 53 - Landing Zones, Organization Policy, and Governance at Scale

Resource hierarchy, identity, networking, policy, project factories, exceptions, and evidence

Learning Objectives

- Design a modular landing zone spanning identity, hierarchy, networking, security, logging, and billing.
- Use organization policies, tags, folders, and project factories for scalable governance.
- Separate platform, security, network, billing, and workload responsibilities.
- Operate exceptions, policy simulation, drift, onboarding, and lifecycle at enterprise scale.

Why This Matters in Production

Landing Zones, Organization Policy, and Governance at Scale is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

53.1 Landing Zone Architecture

A Google Cloud landing zone, or cloud foundation, is a modular and scalable configuration that enables secure adoption. It includes organization and billing setup, identity onboarding, resource hierarchy, network topology, security controls, centralized logging, key management, project creation, and operational ownership. It is not a static one-time deployment; it evolves as workloads, regulations, and organization structures change.

53.2 Resource Hierarchy and Delegation

Folders should express policy and administration boundaries, not an attempt to mirror every department chart. Common dimensions include environment, workload class, regulatory scope, and lifecycle. Use groups rather than individual IAM grants. Separate organization policy, billing, network, security, and workload administrators. Limit organization-level roles and test recovery if a critical admin group is unavailable.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

53.3 Organization Policy and Project Factory

Organization Policy constrains resource configuration through inherited constraints. Custom constraints extend governance but require careful simulation and service support review. A project factory creates projects with owners, labels/tags, APIs, billing, budgets, service identities, logs, network attachment, and deletion workflow. Project creation should be fast enough that teams do not reuse unsafe shared projects.

53.4 Exceptions, Evidence, and Lifecycle

Policy exceptions need business justification, compensating controls, scope, owner, approval, expiration, and review. Use Policy Simulator and canary folders before enforcing broad changes. Export policies, IAM, inventory, findings, and logs as evidence. Offboarding must disable workloads, preserve required data, revoke access, remove network routes, settle billing, and delete projects through a controlled retention process.

Production Design Checklist

- Design a modular landing zone spanning identity, hierarchy, networking, security, logging, and billing.
- Use organization policies, tags, folders, and project factories for scalable governance.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.

- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Policy blocks new managed service	Constraint not evaluated against service prerequisites	Use simulator/canary folder, document exception, and update blueprint after review
Project factory is slow	Manual approval, shared queue, or too many bespoke options	Automate standard path, parallelize safe controls, and separate rare exceptions
Inherited role is unexpected	Folder hierarchy or group nesting not understood	Use Policy Analyzer and resource hierarchy export; correct at appropriate ancestor
Deleted project needed for evidence	Lifecycle lacks retention and legal/compliance review	Implement hold/recovery window, export evidence, and require deletion approval

Hands-on Lab 53: Design and Test a Governed Landing-Zone Slice

Prerequisites

- A sandbox organization or representative folder structure, Terraform, organization policy access, billing controls, and central logging.

Implementation

1. Create an architecture decision for identity, hierarchy, network, security, logging, and billing ownership.
2. Implement a small folder hierarchy for platform, security, non-production, and production workloads.
3. Build a project-factory module with mandatory owner, environment, data classification, budget, logs, and API inputs.
4. Apply one safe organization policy at a canary folder and test compliant and noncompliant resources.
5. Create an exception object with expiry and compensating control.
6. Export effective IAM, organization policies, assets, and logs as an evidence package.
7. Run project offboarding through disable, retention, access revocation, and deletion approval.

Validation Evidence

- The project factory produces a usable governed project without manual edits.
- Policy inheritance and exceptions are visible and explainable.
- Administrative duties are separated.
- Onboarding and offboarding evidence is reproducible.

Cleanup

- Remove sandbox folders, projects, policies, exceptions, billing links, and exported sensitive evidence.

Interview Questions and Model Answers

1. What is a landing zone?

A modular cloud foundation covering identity, hierarchy, networking, security, logging, billing, and operational controls for scalable adoption.

2. Why not mirror the organization chart in folders?

Business structures change and may not align with policy, security, environment, or lifecycle boundaries.

3. What belongs in a project factory?

Ownership, metadata, billing, budgets, APIs, identities, logging, network, policy, lifecycle, and evidence controls.

4. How should exceptions be managed?

As time-bound risk records with precise scope, approval, compensating controls, monitoring, and expiry.

Chapter Summary

- Landing zones are evolving operating systems for cloud adoption.
- Hierarchy, identity, network, security, and billing decisions must be designed together.
- A fast project factory reduces pressure to create unsafe shared environments.
- Policy changes and exceptions require simulation, canary scope, and evidence.

Official Google Cloud References

<https://docs.cloud.google.com/architecture/landing-zones>

<https://docs.cloud.google.com/architecture/landing-zones/decide-resource-hierarchy>

<https://docs.cloud.google.com/resource-manager/docs/organization-policy/overview>

Chapter 54 - Well-Architected, Reliability Engineering, Sustainability, and FinOps

Architecture reviews, resilience, capacity, cost allocation, commitments, carbon, and continuous improvement

Learning Objectives

- Apply the Google Cloud Well-Architected Framework as an evidence-based review process.
- Translate business impact into availability, recovery, performance, and capacity requirements.
- Implement FinOps allocation, budgets, anomaly response, optimization, and commitment governance.
- Include sustainability and operational simplicity in architecture decisions.

Why This Matters in Production

Well-Architected, Reliability Engineering, Sustainability, and FinOps is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

54.1 Well-Architected Review

A review is not a checklist score. It tests whether business requirements, architecture, operations, security, reliability, performance, cost, and sustainability are aligned and evidenced. Record risks, owners, due dates, decisions, and accepted trade-offs. Review before launch, after major change, after significant incident, and periodically for critical services.

54.2 Reliability and Resilience

Define service-level objectives, failure domains, recovery time objective, recovery point objective, dependency assumptions, and degraded modes. Backups are useful only after successful restore tests. Regional or multi-regional design does not guarantee application recovery if identity, DNS, keys, quotas, deployment pipelines, or operational access remain single points of failure. Conduct game days with observable hypotheses and stop conditions.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

54.3 FinOps Operating Model

FinOps creates shared accountability between engineering, finance, and business. Allocate cost using projects, labels, tags, billing export, and service ownership. Monitor budgets and anomalies, but do not treat budgets as hard service limits. Optimize architecture, idle resources, data retention, egress, logging, and workload scheduling before purchasing commitments. Commitment decisions require stable baseline demand and governance for unused coverage.

54.4 Sustainability and Simplicity

Sustainability improves when workloads use efficient managed services, right-sized capacity, appropriate regions, autoscaling, batch scheduling, storage lifecycle, and reduced waste. Architectural simplicity lowers operational load and often reduces both cost and emissions. Measure unit economics such as cost per tenant, request, pipeline, or dataset so optimization does not accidentally reduce business output or reliability.

Production Design Checklist

- Apply the Google Cloud Well-Architected Framework as an evidence-based review process.
- Translate business impact into availability, recovery, performance, and capacity requirements.
- Identity and permissions use least privilege and short-lived credentials.

- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Cost rises without traffic growth	Logging, egress, idle capacity, retention, data scan, or commitment mismatch	Use billing export and unit metrics; attribute change by SKU, project, service, and owner
DR test misses RTO	Manual dependency, quota, data copy, or access delay	Automate runbook, reserve quota, pre-stage dependencies, and repeat timed test
Commitment underutilized	Demand forecast wrong or workload moved	Centralize coverage management, reassign eligible usage, and change purchase governance
Review produces no action	Checklist lacks owners and risk decisions	Create prioritized findings with owner, due date, evidence, and acceptance authority

Hands-on Lab 54: Run a Well-Architected and FinOps Review

Prerequisites

- An existing lab architecture from Parts A-D, billing export or sample cost data, monitoring, backup, and architecture documentation.

Implementation

1. Define business service, users, critical journeys, SLO, RTO, RPO, data classification, and cost objective.
2. Map architecture, dependencies, failure domains, identities, data flows, and operational owners.
3. Review operational excellence, security, reliability, performance, cost, and sustainability evidence.
4. Query billing data by service and owner; calculate one unit-cost metric.
5. Select one reliability risk and conduct a bounded game day or restore test.
6. Select one cost optimization and validate that it does not violate SLO, recovery, or security.
7. Publish decisions, accepted risks, owners, due dates, and next review trigger.

Validation Evidence

- Review findings cite evidence rather than opinion.
- Restore or failure test records measured recovery.
- Cost optimization includes unit economics and reliability guardrails.
- Every material risk has an owner and disposition.

Cleanup

- Remove test load, temporary backups, dashboards, exports, and resources while preserving sanitized review evidence.

Interview Questions and Model Answers

1. What is the purpose of a Well-Architected review?

To identify and own evidence-based risks and trade-offs across business and technical dimensions, not to produce a perfect score.

2. Why are budgets not hard limits?

They notify about spend but generally do not stop resources safely; automated shutdown can cause business or data loss.

3. When should commitments be purchased?

After rightsizing and with stable, measurable baseline demand plus governance for coverage and renewal.

4. How does sustainability relate to DevOps?

Efficient architecture, autoscaling, scheduling, reduced waste, and measured utilization are operational engineering practices that also reduce environmental impact.

Chapter Summary

- Architecture review converts assumptions into owned evidence and decisions.
- Reliability requires recovery tests across technical and operational dependencies.
- FinOps measures unit value and shared accountability, not only lower bills.
- Simplicity and efficient utilization improve operations, cost, and sustainability.

Official Google Cloud References

<https://cloud.google.com/architecture/framework>

<https://cloud.google.com/docs/cloud-finops>

<https://cloud.google.com/sustainability>

Chapter 55 - Enterprise Google Cloud Platform and Secure Delivery Capstone

Integrating landing zones, platform products, CI/CD, observability, security, governance, recovery, and cost controls

Learning Objectives

- Design an enterprise Google Cloud platform with clear ownership and trust boundaries.
- Build a compliant golden path from project creation to monitored production deployment.
- Prove security, reliability, recovery, audit, and FinOps controls through evidence.
- Run realistic failure, policy, and incident scenarios and improve the platform.

Why This Matters in Production

Enterprise Google Cloud Platform and Secure Delivery Capstone is an operating capability, not a console feature. Production teams must define ownership, trust boundaries, failure behavior, change control, evidence, cost, and recovery. The following sections connect the technology to those operating decisions.

55.1 Scenario and Requirements

A regulated digital-service company needs multiple product teams to deploy containerized services and data workloads. It requires isolated development and production, keyless CI/CD, centrally governed networking, service perimeters for sensitive data, organization-level evidence, SLOs, recovery objectives, cost allocation, and a security-operations workflow. The platform team must deliver a supported golden path without becoming a manual ticket queue.

55.2 Target Architecture

Use an organization and folder hierarchy with separated platform, security, network, logging, and workload projects. A project factory creates governed projects and Shared VPC attachment. Terraform modules and Infrastructure Manager or controlled runners manage foundation and workload infrastructure. Cloud Build creates immutable artifacts; Cloud Deploy promotes them; Workload Identity Federation and Binary Authorization control trust. Monitoring, Logging, Asset Inventory, Security Command Center, and Google SecOps provide operations and evidence.

Production principle: Prefer controls that are versioned, testable, observable, reversible, and explainable to the team that must operate them during an incident.

55.3 Control and Failure Design

Define identity and policy owners, state and artifact recovery, build and deploy SLOs, application SLOs, log retention, key lifecycle, perimeter exceptions, finding SLAs, and cost allocation. Test at least five scenarios: destructive Terraform plan, compromised pipeline claim, failed canary, telemetry outage, and perimeter denial. Add a restore or regional recovery test and a security finding that requires coordinated containment.

55.4 Acceptance and Portfolio Evidence

Acceptance is based on reproducible evidence: architecture diagrams, ADRs, module tests, plan and approval records, artifact provenance, policy denials, rollout health, SLO and alert results, asset and audit exports, key rotation, finding lifecycle, incident timeline, restore timing, unit cost, and cleanup. Redact identifiers and secrets before using evidence in a portfolio.

Production Design Checklist

- Design an enterprise Google Cloud platform with clear ownership and trust boundaries.

- Build a compliant golden path from project creation to monitored production deployment.
- Identity and permissions use least privilege and short-lived credentials.
- Change plans include preview, approval, canary scope, rollback, and evidence.
- Metrics, logs, audit records, ownership, cost, and retention are defined.
- Failure and recovery are tested rather than assumed.

Troubleshooting Matrix

Symptom	Likely causes	Engineering response
Golden path requires tickets	Platform product lacks automation or delegated boundaries	Automate project and service products; reserve tickets for exceptions and risk approvals
Security blocks delivery repeatedly	Controls introduced without dependency mapping or feedback	Use preview/dry-run, canary scope, clear errors, and module-integrated fixes
Evidence is incomplete	Logs, approvals, or ownership not designed as requirements	Create evidence matrix and test collection in each lab scenario
Recovery works only for platform team	Runbook, access, or tooling is not transferable	Use role-based drills, documented break-glass, and independent operator validation

Hands-on Lab 55: Build and Defend an Enterprise Secure Delivery Platform

Prerequisites

- Sandbox organization or representative projects, Terraform, CI/CD, Cloud Run or GKE, observability, security, billing export, and documentation repository.

Implementation

1. Publish requirements, threat model, ownership matrix, ADRs, SLOs, RTO/RPO, data classification, and cost objectives.
2. Deploy the landing-zone slice and project factory with policies, central logs, budgets, and network attachment.
3. Publish a versioned workload module with identity, service perimeter compatibility, monitoring, and secure defaults.
4. Build an application through keyless CI, record provenance, and enforce deploy policy.
5. Promote through Cloud Deploy with a production canary and automated verification.
6. Operate SLOs, alerts, logs, traces, inventory, posture findings, and a security case.
7. Run destructive-plan, failed-canary, perimeter-denial, telemetry-loss, credential-compromise, and restore scenarios.
8. Measure recovery, alert quality, finding workflow, build/deploy reliability, and unit cost; then implement prioritized improvements.
9. Create a sanitized evidence pack and conduct an architecture and incident review.

Validation Evidence

- A new team can obtain a compliant environment through the supported path.
- Untrusted source or artifact cannot reach production.
- Service health and security events are detected with actionable context.
- Recovery and rollback meet measured objectives.
- Cost, ownership, policy, and evidence are attributable.

Cleanup

- Destroy all lab projects and resources through controlled workflows after preserving only sanitized portfolio evidence and required audit records.

Interview Questions and Model Answers

1. What proves a platform is production-ready?

Demonstrated secure onboarding, reliable delivery, actionable observability, governed exceptions, incident response, recovery, cost ownership, and evidence under realistic failure.

2. Why is a golden path more than a template?

It includes infrastructure, identity, security, delivery, observability, documentation, support, lifecycle, and measured service quality.

3. How should capstone acceptance be decided?

Against explicit functional, security, reliability, recovery, operational, evidence, and cost criteria rather than deployment success alone.

4. What is the strongest portfolio evidence?

Sanitized artifacts that show decisions, implementation, failure testing, measured results, incident learning, and improvements.

Chapter Summary

- The capstone integrates the full enterprise Google Cloud operating model.
- A platform is a supported product with security, reliability, recovery, and cost obligations.
- Failure injection and incident scenarios reveal gaps that normal deployment does not.
- Acceptance requires measurable evidence and accountable ownership.

Official Google Cloud References

<https://docs.cloud.google.com/architecture/landing-zones>

<https://cloud.google.com/architecture/framework>

<https://docs.cloud.google.com/security-command-center/docs/security-command-center-overview>

Enterprise Practice Projects

These projects extend Chapters 41-55 into portfolio-grade exercises. Each project should include requirements, architecture, threat model, ADRs, source, automated tests, deployment evidence, dashboards, runbooks, failure tests, cost notes, cleanup, and a short retrospective.

Project	Expected outcome
Governed Project Factory	Create a reusable Terraform product that provisions workload projects with ownership, APIs, logging, budgets, organization-policy compatibility, Shared VPC attachment, and lifecycle controls.
Keyless Multi-Repository CI Platform	Federate two external repositories into Google Cloud with distinct attribute conditions, build identities, Artifact Registry permissions, and audit evidence.
Progressive Cloud Run Delivery	Build once, promote by digest, canary with Cloud Deploy, verify SLOs, and demonstrate rollback plus expand-contract data migration.
GKE Supply-Chain Enforcement	Combine Artifact Registry, provenance, attestations, Binary Authorization, Workload Identity Federation for GKE, and incident revocation.
Central Observability Project	Create multi-project metrics scope, structured log routing, OpenTelemetry collection, SLOs, burn-rate alerts, and telemetry-cost controls.
Audit and Asset Compliance Lake	Export audit logs and Cloud Asset Inventory to central storage/BigQuery, detect ownership gaps, and create reversible remediation workflow.
Cryptographic Lifecycle Service	Implement Secret Manager, CMEK, key rotation, certificate expiry monitoring, separation of duties, and recovery from disabled key version.
Security Finding Operations	Route Security Command Center findings to owners, enrich risk, automate one safe remediation, track exceptions, and verify closure.
Data Exfiltration Guardrail	Use VPC Service Controls dry-run, restricted Google APIs, access levels, and exception workflow around a sensitive data project.
Enterprise Landing-Zone Review	Assess a landing zone against Well-Architected, reliability, recovery, FinOps, and sustainability requirements; execute one game day and cost optimization.

Senior Interview Drill Bank

1. A Terraform plan shows replacement of a production database. What do you do?

Stop. Determine the immutable property and state identity, validate backup and migration, use import/moved operations if appropriate, obtain explicit approval, and test in a representative environment.

2. How would you prevent CI credential theft?

Use Workload Identity Federation with strict issuer, audience, repository, branch, and environment conditions; separate identities; short sessions; no static keys; and audit/alert on impersonation.

3. How do you design a useful canary?

Choose representative traffic or capacity, meaningful bake time, segmented user and business indicators, automated stop conditions, and a tested rollback or roll-forward path.

4. What is the difference between logging and audit evidence?

Application logs explain software behavior; audit logs record control-plane and data-access actions. They differ in source, trust, retention, access, and investigation use.

5. How do you reduce alert fatigue?

Page only on actionable service impact, use SLO burn rates, group dependent symptoms, suppress maintenance, route diagnostics separately, and review false positives after incidents.

6. How do you respond to an exposed artifact-signing identity?

Freeze releases, revoke identity/federation, invalidate or distrust attestations, inventory affected digests and runtimes, rebuild with a clean trust chain, and preserve evidence.

7. Why can VPC Service Controls break pipelines?

Pipelines often cross projects, identities, APIs, consoles, and external systems. Perimeter ingress, egress, access levels, DNS, and supported-service limits must be mapped and tested in dry-run.

8. How do you apply least privilege safely?

Use Policy Analyzer and usage evidence, simulate changes, reduce access in canary scope, monitor Policy Denied logs, and retain a time-bound recovery path.

9. What should a project factory enforce?

Ownership, environment and data classification, billing, budgets, APIs, identities, logging, network, policy, retention, evidence, and deletion workflow.

10. How do you prove backups are useful?

Restore them into an isolated environment, validate application and data consistency, measure recovery time and point, and test operator access and dependencies.

11. How would you prioritize security findings?

Combine severity with exposure, privilege, exploitability, active threat, attack path, business criticality, data sensitivity, and compensating controls.

12. What is unit economics in cloud operations?

Cost normalized by a business or engineering output, such as cost per tenant, request, pipeline run, dataset, or transaction.

13. Why is a managed service not automatically secure?

The provider manages service infrastructure, while the customer still owns configuration, identity, data, integration, monitoring, and operational use.

14. How do you make platform engineering successful?

Treat the platform as a product: narrow catalog, secure defaults, version contracts, documentation, support, SLOs, feedback, adoption metrics, and deprecation.

15. What should a production evidence pack contain?

Architecture and ADRs, reviewed plans, test results, artifact digest/provenance, approvals, policy decisions, rollout health, SLOs, audit/inventory, incident and recovery results, cost, and ownership.

Google Cloud DevOps Command Reference

Area	Representative commands
Terraform quality	<pre>terraform fmt -check -recursive terraform init terraform validate terraform plan -out=tfplan terraform show -json tfplan > plan.json</pre>
Infrastructure Manager	<pre>gcloud infra-manager previews create PREVIEW --location=REGION --service-account=SA --local-source= gcloud infra-manager deployments create DEPLOYMENT -- location=REGION --service-account=SA --local-source=</pre>
Cloud Build	<pre>gcloud builds submit --config=cloudbuild.yaml . gcloud builds list --limit=20 gcloud builds log BUILD_ID</pre>
Cloud Deploy	<pre>gcloud deploy releases create RELEASE --delivery- pipeline=PIPELINE --region=REGION --images=APP=IMAGE_DIGEST gcloud deploy rollouts list --delivery-pipeline=PIPELINE -- release=RELEASE --region=REGION</pre>
Monitoring	<pre>gcloud monitoring policies list gcloud monitoring dashboards list gcloud services list --enabled --filter=monitoring</pre>
Logging and audit	<pre>gcloud logging sinks list gcloud logging read "logName:cloudaudit.googleapis.com" --limit=20 gcloud logging buckets list --location=global</pre>
Asset Inventory	<pre>gcloud asset search-all-resources --scope=organizations/ORG_ID -- query="NOT labels.owner:*" gcloud asset export --organization=ORG_ID --output-path=gs://BUCKET/assets.json</pre>
Policy Intelligence	<pre>gcloud policy-intelligence troubleshoot-policy iam RESOURCE -- principal-email=PRINCIPAL --permission=PERMISSION gcloud recommender recommendations list --project=PROJECT -- location=global --recommender=google.iam.policy.Recommender</pre>
Secret Manager	<pre>gcloud secrets versions add SECRET --data-file= gcloud secrets versions access latest --secret=SECRET gcloud secrets versions disable VERSION --secret=SECRET</pre>
Cloud KMS	<pre>gcloud kms keyrings list --location=REGION gcloud kms keys versions list --key=KEY --keyring=RING -- location=REGION</pre>
Security Command Center	<pre>gcloud scc findings list ORGANIZATION_ID --source=SOURCE_ID gcloud scc assets list ORGANIZATION_ID</pre>
VPC Service Controls	<pre>gcloud access-context-manager perimeters list --policy=POLICY_ID gcloud access-context-manager levels list --policy=POLICY_ID</pre>

Command safety: Replace placeholders deliberately. Confirm active account, project, region, organization, and target resource before any mutating command. Use `--format` or `--dry-run/preview` features where available and capture output as evidence.

Production Readiness Checklist

Infrastructure as Code

- ☐ State is protected, versioned, recoverable, and has one active writer per stack.
- ☐ Providers and modules are pinned and upgraded through tested release procedures.
- ☐ Plans are reviewed for replacement and destroy actions.
- ☐ Imports, refactors, drift, and emergency recovery are documented.

CI/CD and Supply Chain

- ☐ Pipelines use keyless short-lived authentication and separate build/deploy identities.
- ☐ Artifacts are immutable by digest with provenance and required attestations.
- ☐ Production targets enforce policy and have controlled break-glass.
- ☐ Canary, rollback/roll-forward, schema compatibility, and release evidence are tested.

Observability

- ☐ Service ownership, SLIs, SLOs, error budgets, dashboards, and runbooks exist.
- ☐ Alerts are actionable, routed, grouped, and reviewed for quality.
- ☐ Logs and traces are structured, correlated, privacy-aware, and cost-controlled.
- ☐ Telemetry pipeline failure and source freshness are monitored.

Security and Cryptography

- ☐ Secrets and keys use least privilege, rotation, lifecycle separation, and recovery tests.
- ☐ Security findings have owners, SLAs, exceptions, and verified closure.
- ☐ Security telemetry ingestion and detection coverage are monitored.
- ☐ Perimeters, edge controls, and firewall policies use preview/dry-run and canary rollout.

Governance and Operations

- ☐ Landing-zone ownership and delegated responsibilities are documented.
- ☐ Project factory produces compliant projects quickly.
- ☐ Organization-policy changes are simulated and canaried.
- ☐ Audit logs, asset inventory, policy evidence, and retention meet requirements.

Reliability and FinOps

- ☐ RTO/RPO, failure domains, backup/restore, game days, and break-glass are tested.
- ☐ Costs are allocated to owners and measured with unit economics.
- ☐ Budgets, anomalies, idle resources, retention, egress, and commitments are governed.
- ☐ Architecture reviews create owned findings and accepted trade-offs.

Glossary

Term	Meaning
Artifact attestation	A signed statement about an immutable artifact, such as approval or policy compliance.
Binary Authorization	A deploy-time policy control that evaluates artifact evidence before allowing supported workloads to run.
Blueprint	A reusable Terraform configuration or composed infrastructure design; Infrastructure Manager uses the term for Terraform configuration.
Burn rate	The speed at which a service consumes its error budget.
CMEK	Customer-managed encryption key controlled through Cloud KMS for supported service data.
Cloud Asset Inventory	A global metadata inventory used to search, export, monitor, and analyze Google Cloud assets.
Cloud Deploy	Managed continuous-delivery service for promoting releases to deployment targets.
Error budget	The amount of unreliability allowed by an SLO during a measurement period.
Golden path	A supported, low-friction platform route that produces compliant, operable workloads.
Hermetic build	A build whose inputs and toolchain are explicitly controlled, minimizing hidden host or network dependencies.
Infrastructure Manager	Managed Google Cloud service for previewing and deploying Terraform blueprints.
Key version	A cryptographic version inside a Cloud KMS key; rotation creates a new primary version.
Landing zone	A modular cloud foundation covering hierarchy, identity, networking, security, logging, billing, and operations.
Metric scope	A Cloud Monitoring configuration that lets one scoping project view metrics from multiple projects.
OpenTelemetry	Open standard and tooling for generating, collecting, and exporting metrics, logs, and traces.
Policy Analyzer	Tooling that answers which principals have what access to which resources based on policies.
Policy Simulator	Tooling that predicts the access or organization-policy effect of proposed changes.
Provenance	Evidence describing how an artifact was built, including source, builder, parameters, and output digest.
Service perimeter	A VPC Service Controls boundary around supported Google-managed services and resources.
SIEM	Security information and event management for collecting, searching, correlating, and detecting across security telemetry.
SLI	A measured indicator of service reliability or user experience.
SLO	A target for an SLI over a defined period.
SOAR	Security orchestration, automation, and response for investigation and response workflows.
Terraform state	The coordination record mapping Terraform resource addresses to managed remote objects and attributes.
Unit economics	Cost normalized by a business or engineering output.
Workload Identity Federation	Exchange of external identity credentials for short-lived Google Cloud access without service-account keys.

Official Google Cloud Documentation Index

Topic	Official documentation
Terraform on Google Cloud	https://docs.cloud.google.com/docs/terraform
Infrastructure Manager	https://docs.cloud.google.com/infrastructure-manager/docs
Terraform blueprints	https://docs.cloud.google.com/docs/terraform/blueprints/terraform-blueprints
Cloud Build	https://docs.cloud.google.com/build/docs
Cloud Deploy	https://docs.cloud.google.com/deploy/docs
Workload Identity Federation for deployment pipelines	https://docs.cloud.google.com/iam/docs/workload-identity-federation-with-deployment-pipelines
Binary Authorization	https://docs.cloud.google.com/binary-authorization/docs
Cloud Monitoring	https://docs.cloud.google.com/monitoring/docs
Managed Service for Prometheus	https://docs.cloud.google.com/stackdriver/docs/managed-prometheus
Google Cloud Observability	https://docs.cloud.google.com/stackdriver/docs
Google-built OpenTelemetry Collector	https://docs.cloud.google.com/stackdriver/docs/instrumentation/google-built-otel
Cloud Logging	https://docs.cloud.google.com/logging/docs
Cloud Audit Logs	https://docs.cloud.google.com/logging/docs/audit
Cloud Asset Inventory	https://docs.cloud.google.com/asset-inventory/docs
Policy Intelligence	https://docs.cloud.google.com/policy-intelligence/docs
Secret Manager	https://docs.cloud.google.com/secret-manager/docs
Cloud KMS	https://docs.cloud.google.com/kms/docs
Security Command Center	https://docs.cloud.google.com/security-command-center/docs
Google Security Operations	https://docs.cloud.google.com/chronicle/docs
VPC Service Controls	https://docs.cloud.google.com/vpc-service-controls/docs
Cloud Armor	https://docs.cloud.google.com/armor/docs
Landing zones	https://docs.cloud.google.com/architecture/landing-zones
Google Cloud Well-Architected Framework	https://cloud.google.com/architecture/framework
Cloud FinOps	https://cloud.google.com/docs/cloud-finops

End of Volume 8E

Next: Volume 8F completes Google Cloud with enterprise architecture, migration, incident response, resilience, performance, FinOps, platform engineering, advanced troubleshooting, certifications, interviews, and the final integrated capstone.